```
      minDistance ← distance(childNode.centroid,
point)                          #7
      result     ←
childNode                                                  #8
  return
result
      #9
```

#1 Method `findClosestChild` is defined on `SsNode`. It takes a point `target` and returns the child of current node whose distance to `target` is minimal.
#2 If we call this method on a leaf, there is something wrong. In some languages, we can use assert to make sure the invariant (`not node.leaf`) is true.
#3/#4 Properly initialize the minimum distance and the node that will be returned. Another implicit invariant is that an internal node has at least 1 child (there must be at least `m`), so these values will be updated at least once.
#5 Cycle through all children.
#6 Check if the distance between the current child's centroid and `target` is smaller than the minimum found so far.
#7/#8 If it is so, store the new minimum distance, and update the closest node.
#9 After the `for` loop cycled through all children, we can return the closest one found.

If you refer to figure 10.16, you can see how a split along the `x` axis would have produced two sets with points `G` and `H` on one side, and `F` and `Z` on the other: comparing the result with figure 10.14, there is no doubt about what's the best final result!